

REPUBLIQUE FRANCAISE

Donnees ouvertes - Ville de Paris - OpenStreetMap - data.gouv.fr

Urban Data Explorer

Observatoire socio-urbain de Paris - Architecture médaillon

Rapport de soutenance M1 - Data Engineering & IA

RNCP40875 - Bloc 1 - Expert en Ingénierie des Données

Certificateur : Efrei - Paris Panthéon-Assas Université

Binôme : Adam Beloucif & Emilien Morice

Année 2025-2026

CONFIDENTIEL PEDDA - ne pas diffuser hors jury



Table des matieres

1. Contexte et problematique
2. Organisation du binome et methode de travail
3. Architecture d'ensemble : le pattern medaillon
4. Stack technique
5. Le catalogue des sources : 83 jeux de donnees en 4 familles
6. Couverture OpenStreetMap : Overpass API
7. Pipeline ETL : de la source brute au datamart
8. Auto-ingestion hebdomadaire (Task Scheduler)
9. C1.1 - Base relationnelle PostgreSQL
10. C1.2 - Base NoSQL Cassandra
11. C1.3 / C1.4 - Data Lake Parquet et scalabilite
12. C2.1 - API REST FastAPI
13. C2.2 - Streaming Kafka temps reel
14. C2.3 / C2.4 - Transformation et optimisation
15. Indicateurs Gold et score de synthese
16. Qualite et tracabilite
17. Interface DSFR et accessibilite
18. Resultats et validation
19. Couverture de la grille RNCP40875
20. Conclusion et perspectives

Resume executif

Urban Data Explorer est une plateforme d'ingenierie des donnees qui collecte, normalise et restitue 83 jeux de donnees ouvertes sur Paris, en quatre familles thematiques, pour produire un observatoire socio-urbain comparatif par arrondissement.

83

sources integrees

4

familles

20

arrondissements

118

tests passes

Le projet met en oeuvre une architecture medaillon complete (Bronze, Silver, Gold), une API REST documentee, un streaming Kafka, une base relationnelle PostgreSQL et une base NoSQL Cassandra. Il couvre l'ensemble des competences C1.1 a C2.4 du Bloc 1 RNCP40875.

Les donnees immobilieres (DVF 2023 - data.gouv.fr) et de revenus (INSEE Filosofi 2020) alimentent les indicateurs Gold avec des valeurs reelles. L'auto-ingestion hebdomadaire (Windows Task Scheduler, XML natif, sans .bat ni .ps1) garantit la fraicheur du catalogue chaque dimanche.

1 Contexte et problematique

La Ville de Paris publie des centaines de jeux de données ouvertes sur opendata.paris.fr, data.gouv.fr et data.education.gouv.fr. Ces fichiers couvrent le logement social, la mobilité, la qualité de vie, l'environnement, les services publics, les commerces, et bien d'autres dimensions de la vie urbaine. Le problème : ils sont dispersés, dans des formats différents (CSV, TSV, GeoJSON), avec des références géographiques hétérogènes (code postal, code INSEE, coordonnées GPS, code IRIS).

Un décideur - maire d'arrondissement, urbaniste, chercheur - qui veut comparer deux quartiers sur cinq indicateurs doit aujourd'hui assembler manuellement des dizaines de fichiers. Ce projet répond à ce besoin en construisant une chaîne de données qui fait ce travail automatiquement et produit un observatoire cartographique prêt à l'emploi.

La question centrale

Comment concevoir une architecture de données robuste, évolutive et pertinente pour produire un observatoire socio-urbain de Paris à partir de sources hétérogènes publiques, en couvrant l'ensemble des compétences d'architecture de stockage et de traitement du Bloc 1 RNCP40875 ?

Perimetre

- **Géographique** : Paris intra-muros, 20 arrondissements, 80 quartiers IRIS.
- **Thématique** : mobilité, vie quotidienne (éducation, santé, commerce, culture), environnement, logement et urbanisme.
- **Temporel** : données millésimées 2020-2024 ; flux temps réel Velib' et chantiers.
- **Technique** : ingestion, stockage multi-modèle, traitement Polars, API REST, streaming Kafka, interface cartographique.

2 Organisation du binome et methode de travail

Le projet a ete conduit en binome (Adam Beloucif & Emilien Morice) sur toute la duree du semestre. La methode choisie est iterative : d'abord faire circuler la donnee de bout en bout avec un catalogue minimal, puis etendre le catalogue, enrichir les transformations et soigner l'interface.

Repartition des roles

- **Ingestion & ETL** : conception du catalogue SourceSpec, pipeline Bronze/Silver/Gold, geocodage hors ligne, auto-ingestion Task Scheduler.
- **Base de donnees** : schema en etoile PostgreSQL, modelisation Cassandra, tests de charge.
- **API & streaming** : routeurs FastAPI, auth JWT, quotas par IP, producteur/consommateur Kafka.
- **Interface** : dashboard React/TypeScript, carte MapLibre (fond IGN), theme DSFR clair/sombre.

Methode agile simplifiee

Chaque iteration commence par une revue du catalogue (quelles sources manquent, quels indicateurs sont faux) et se termine par un commit granulaire et un test de fumee de l'API. Le versionnage Git avec branche feat/* + pull request assure la tracabilite de chaque decision.

Environnements

- **Local-first** : le projet tourne sur un simple poste sans Docker ni base externe, grace aux fallbacks Parquet et aux constantes de reference.
- **Distribue** : Docker Compose avec 11 services (api, postgres, cassandra, kafka, zookeeper, hadoop, hive, spark, frontend, nginx, prometheus).
- **CI** : 118 tests pytest passes a chaque push ; pas de secrets commites (.env gitignored).

3 Architecture d'ensemble : le pattern medaillon

Le projet suit l'architecture medaillon, un pattern de reference en ingenierie des donnees qui organise les donnees en trois couches successives. Chaque couche correspond a un niveau de maturite de la donnee.

Bronze - la donnee brute

Les fichiers sont telecharges tels quels depuis les sources officielles et stockes dans `data/raw/downloads/`. Un fichier Bronze est une photographie exacte de la source a un instant t. Il n'est jamais modifie apres ingestion. Format : CSV, TSV (Overpass), ou CSV.GZ (gros jeux data.gouv.fr).

Silver - la donnee normalisee

Le pipeline ETL (Polars) transforme chaque fichier Bronze en un enregistrement normalise appele un 'record Silver'. Chaque record contient : `source_id`, `family`, `arrondissement_code` (normalise 75001-75020), `latitude`, `longitude`, `code_iris`, `nom_iris`, `address`, `quality_flag`. Le geocodage est realise hors ligne par point-in-polygon sur les contours IRIS (GeoJSON officiel IGN), sans aucun appel reseau externe lors du traitement.

Gold - le datamart

A partir du Silver, on construit deux datamarts : le dashboard (une ligne par arrondissement, avec tous les indices calcules) et la timeline (20 arrondissements x 12 mois = 240 lignes). Ces datamarts sont stockes en Parquet (local-first) et charges dans PostgreSQL (mode distribue). Ils alimentent directement l'API et l'interface cartographique.

Schema de flux

Source Open Data -> Bronze (CSV/TSV raw) -> Silver (Polars, geocodage IRIS, normalisation) -> Gold (datamarts Parquet / PostgreSQL) -> API FastAPI -> Interface React/MapLibre

En parallele, un flux temps reel (Kafka) collecte les evenements de disponibilite Velib' et de chantiers perturbants, les achemine dans Cassandra, et les expose via le routeur `/events` de l'API.

4 Stack technique

Couche	Technologie	Role
Traitement	Python 3.12 + Polars	ETL vectorise (moteur Rust), transformations Silver/Gold
Base relationnelle	PostgreSQL 15	Datamart dashboard en schema etoile, tests de charge
Base NoSQL	Cassandra 4	Stockage evenements temps reel, TTL 7 jours, partitionnement
Data Lake	Parquet + HDFS	Couches Bronze/Silver/Gold, columnar, compresse (Snappy)
Streaming	Apache Kafka + Zookeeper	Pipeline temps reel : Velib, chantiers; producteur/consommateur
API	FastAPI + Pydantic v2	REST, auth JWT HS256, quotas par IP, documentation /docs
Interface	React + TypeScript + MapLibre	Carte 2D (tuiles IGN Geoplateforme), chartjs, theme DSFR
CI/CD	pytest (118 tests)	Couverture complete ETL, API, securite, qualite
Orchestration	Docker Compose 11 services	Mode distribue optionnel, profils streaming & lake
Ingestion auto	Windows Task Scheduler (XML)	Ingestion hebdomadaire, sans .bat ni .ps1 (EDR hospitaliers)

Le choix de Polars (moteur Rust) plutot que pandas s'explique par les performances : Polars est entre 5 et 10 fois plus rapide sur les transformations de type groupby/agg, ce qui est determinant quand on traite 83 sources en une passe hebdomadaire. La memoire est egalement mieux geree grace au lazy evaluation plan.

5 Le catalogue des sources : 83 jeux de données en 4 familles

L'ingénierie du catalogue est au cœur du projet. Chaque source est définie par un objet SourceSpec (dataclass Python immutable) qui encapsule l'identifiant, le titre, la famille, l'URL de téléchargement, le fournisseur, le séparateur, l'encodage, et les hints de géocodage (colonnes latitude/longitude, geo_point, arrondissement, adresse).

83 sources - 4 familles - 3 fournisseurs principaux (opendata.paris.fr, data.gouv.fr, OpenStreetMap Overpass) - 30 sources OSM (Overpass API)

Mobilité & Accessibilité

17 sources

- Velib stations + disponibilité temps réel
- Aménagements cyclables (pistes, bandes)
- Stationnement voie publique (points et emprises)
- Bornes recharge électrique IRVE (Belib)
- Points d'arrêt IDFM (Ile-de-France Mobilités)
- Comptages vélos permanents, WiFi gratuit Paris
- Chantiers perturbants, voirie parisienne
- OSM : bus, metro, tramway, vélos en libre-service, taxi, parking

Vie quotidienne

42 sources

- Education : écoles mat./elem., collèges, lycées (annuaire MENJ), crèches, bibliothèques
- Santé : défibrillateurs, sanisettes, hôpitaux IDF, médecins HAS, pharmacies (OSM)
- Commerce : marchés, Semaest, fontaines à boire, kiosques, équipements sportifs
- Culture : agenda Que Faire à Paris, lieux de tournage, musées de France, lieux de culte (OSM)
- OSM : restaurants, cafés, épiceries, boulangeries, banques, hôtels, coiffeurs, librairies
- OSM : crèches (OSM), gymnases, pompiers, dentistes, médecins généralistes

Environnement & Cadre de vie

12 sources

- Espaces verts (parcs, squares, promenades)
- Jardins partagés, îlots de fraîcheur, aire de jeux
- Îlots de chaleur (vulnérabilité thermique IRIS)
- Arbres de Paris (280 000 + alignement et parcs)
- Bruit - indicateurs Lden 2022
- OSM : arcs cyclables supplémentaires, jardins communautaires

Logement & Urbanisme

12 sources

- Logements sociaux finances (inventaire par arr.)
- DPE - Diagnostics de Performance Énergétique (ADEME)
- Logements vacants taxables LOVAC (DGALN)
- Permis de construire déposés à Paris
- Monuments Historiques (référentiel national)
- DVF - Transactions immobilières 2023 (DGFIP)
- INSEE Filosofi - Revenus par IRIS 2020
- Amiante/SIS (BRGM Georisques), ascenseurs, accessibilité PMR

La conception du catalogue respecte le principe open data pur : uniquement des sources officielles publiques (Ville de Paris, État, OpenStreetMap). Aucun scraping de site privé ou sous paywall. Les 30

sources OSM sont interrogées via l'Overpass API avec un filtre de zone précise (`area["ref:INSEE"="75056"]`) pour limiter le périmètre exactement à la commune de Paris.

6 Couverture OpenStreetMap : Overpass API

OpenStreetMap (OSM) est la base cartographique collaborative la plus complete au monde. Elle documente des points d'interet que les jeux de donnees officiels ne couvrent pas ou couvrent mal : restaurants, pharmacies, lieux de culte, transports de surface, commerces de proximite, bornes de service. Le projet interroge OSM via l'Overpass API, un service de requetage en lecture sur la base OSM.

Protocole technique

Chaque source OSM est definie par une requete Overpass QL stockee dans le champ `download_url` avec le prefixe 'OVERPASS:'. L'auto-ingestion detecte ce prefixe et envoie la requete en POST (les requetes Overpass peuvent dépasser 2 000 caracteres). Le format de sortie est TSV (tabulation-separated) avec en-tete, ce qui simplifie l'import Polars.

Exemple de requete Overpass QL pour les pharmacies :
`[out:csv(;;lat,;;lon,name,amenity,opening_hours,phone;true;"t")][timeout:90];area["ref:INSEE"="75056"]
 ->.p;node[amenity=pharmacy](area.p);out body;`

Le filtre `area["ref:INSEE"="75056"]` est plus fiable que `area[name="Paris"]` car il cible le code INSEE exact de la commune de Paris (75056) plutot que le nom, qui peut correspondre a plusieurs communes en France.

Sources OSM integrees (extrait)

Famille	Source OSM	Filtre amenity/shop
mobilité	Arrets de bus	highway=bus_stop
mobilité	Stations de metro	station=subway
mobilité	Parkings auto	amenity=parking
vie_quotidienne	Pharmacies	amenity=pharmacy
vie_quotidienne	Restaurants	amenity=restaurant
vie_quotidienne	Lieux de culte	amenity=place_of_worship
vie_quotidienne	Epicerie	shop=convenience
vie_quotidienne	Boulangeries	shop=bakery
vie_quotidienne	Banques / DAB	amenity=bank
environnement	Jardins comm.	leisure=garden

7 Pipeline ETL : de la source brute au datamart

Etape 1 - Ingestion Bronze

Le script `auto_ingest.py` parcourt le catalogue `ALL_SOURCES` et telecharge chaque source. Deux modes : HTTP GET (avec retry exponentiel, 3 tentatives) pour les API Opendatasoft et `data.gouv.fr` ; POST Overpass pour les sources OSM. Un rate-limiting de 0,5 s entre chaque source evite de surcharger les API publiques. Chaque fichier est nomme `{source_id}.csv` ou `{source_id}.tsv` dans `data/raw/downloads/`.

Etape 2 - Transformation Silver

La fonction `build_silver_record(row, spec)` transforme chaque ligne CSV en un record Silver. Elle applique dans l'ordre : (1) extraction des coordonnees GPS (colonnes latitude/longitude ou `geo_point 'lat,lon'`), (2) geocodage IRIS par point-in-polygon (polygones IRIS charges en memoire depuis le GeoJSON officiel), (3) normalisation du code arrondissement (75101 -> 75001, 750 -> 75001 etc.), (4) attribution du `quality_flag` selon la precision du rattachement.

- `quality_flag = 'gps_iris'` : rattachement par point-in-polygon sur les contours IRIS.
- `quality_flag = 'code_only'` : code arrondissement fourni directement par la source, sans GPS.
- `quality_flag = 'unresolved'` : ni GPS ni code arrondissement exploitable.

Etape 3 - Agregation Gold

La fonction `build_gold_dashboard(silver_df)` agregue les records Silver par arrondissement et par famille. Elle calcule les 4 comptes de famille (`environnement_count`, `mobilite_count`, `vie_quotidienne_count`, `logement_urbanisme_count`), puis les indices derives (immobilier, logement social, revenu, cadre de vie, environnement) et le score global.

Les valeurs de prix au m2 et de revenu median sont chargees depuis les sources reelles (DVF 2023 et INSEE Filosofi 2020) si les fichiers sont disponibles ; sinon, les constantes de reference (valeurs mesurees sur les donnees 2023) servent de fallback transparent.

Tests de l'ETL

118 tests pytest couvrent l'ensemble du pipeline : `_family_counts`, `build_gold_dashboard` (shape, valeurs dans plage, source reel vs reference), `build_silver_record` (geocodage, normalisation), `_enrich_district_row` (preservation des valeurs reelles, fallback), et les endpoints API (catalog, health, datamarts, auth, quotas).

8 Auto-ingestion hebdomadaire

Pour maintenir le catalogue a jour sans intervention manuelle, un Task Scheduler Windows declenche l'ingestion chaque dimanche a 04h00. La planification est definie dans un fichier XML (tasks/urban_ingest_sunday.xml) importe via la commande schtasks.

```
schtasks /Create /XML tasks\urban_ingest_sunday.xml /TN "AdamBrain\UrbanDataIngestSunday"
```

Contrainte EDR hospitaliere

Le projet est developpe dans un environnement ou Microsoft Defender for Endpoint (MDE) est deploye. Les scripts .bat et .ps1 sont bloques par l'EDR. La solution : action directe python.exe + chemin du script dans le XML Task Scheduler, sans aucune couche PowerShell. Cette contrainte est documentee et respectee dans tout le projet.

Parametres de la tache

Parametre	Valeur
Declencheur	Chaque dimanche a 04h00
Commande	C:\...\Python312\python.exe
Arguments	C:\...\urban-data-explorer\scripts\auto_ingest.py
Duree max	2 heures (PT2H)
Sur batterie	Oui (DisallowStartIfOnBatteries=false)
Instances multiples	IgnoreNew

En mode --dry-run, le script simule l'ingestion sans ecrire de fichier, ce qui permet de valider la configuration du catalogue. En mode normal, il telecharge toutes les sources non metadata_only et journalise le resultat (ok/fail par source).

9 C1.1 - Base relationnelle PostgreSQL

PostgreSQL 15 est le moteur relationnel du projet. Il stocke les datamarts Gold sous la forme d'un schema en etoile concu pour l'analyse par arrondissement. Ce choix correspond a la competence C1.1 : 'Concevoir et implementer une base de donnees relationnelle scalable'.

Schema en etoile

- **fact_arrondissement_dashboard** : table de faits (une ligne par arrondissement) avec tous les indices calcules, prix_m2, revenu_median, score global.
- **dim_arrondissement** : dimension geographique (code INSEE, nom, label, coordonnees centroide).
- **dim_iris** : dimension quartier IRIS (code, nom, code_com).
- **dim_source** : dimension source (source_id, titre, famille, fournisseur).

Regles d'integrite

Le schema impose des cles primaires, des cles etrangeres vers les dimensions, des contraintes NOT NULL sur les colonnes critiques et des contraintes CHECK sur les plages d'indices (0-100). Ces regles ont ete verifiees par les tests d'integrite referentielle du benchmark.

Tests de charge

Ce que nous avons mesure	Resultat
Requetes traitees sans erreur	500 sur 500
Debit moyen	environ 1 190 requetes par seconde
Temps de reponse moyen	1,80 ms
Temps de reponse p95	3,50 ms
Integrite referentielle verifiee	3 sur 3 contraintes PASSED

Le benchmark a ete realise avec 10 threads concurrents sur 500 requetes analytiques (GROUP BY, ORDER BY score DESC, filtres sur les indices). Les index sur les cles etrangeres et sur le score global sont determinants pour maintenir la latence p95 sous 4 ms.

10 C1.2 - Base NoSQL Cassandra

Apache Cassandra est utilise pour stocker les evenements du flux temps reel : disponibilite des stations Velib' et evenements de chantiers. Le choix de Cassandra est justifie par ses caracteristiques : ecritures massives en $O(1)$, modele append-only adapte aux flux d'evenements, et TTL natif pour la purge automatique.

Modelisation orientee requetes

La table `events_by_type` est partitionnee par `event_type` (`velib_update`, `chantier_update`, etc.) et trie par `event_time` DESC dans le clustering. Cette conception garantit que la requete 'donner les N evenements recents d'un type donne' est $O(N)$ sans scan complet.

```
PRIMARY KEY (event_type, event_time DESC, event_id) - TTL 604 800 s (7 jours) - RF=1
SimpleStrategy
```

Complementarite SQL/NoSQL

PostgreSQL et Cassandra coexistent et se complementent. PostgreSQL sert l'analytique structuree (comparaison d'arrondissements, calcul de scores, jointures) ; Cassandra sert le flux d'evenements (derniers snapshots Velib, alertes chantiers). L'API expose les deux via des routeurs distincts (`/datamarts` et `/events`).

11 C1.3 / C1.4 - Data Lake Parquet et scalabilite

C1.3 - Data Lake

Les donnees sont stockees en Parquet, un format colonnaire compresse (codec Snappy) particulierement adapte aux requetes analytiques. Chaque couche (Bronze, Silver, Gold) est un repertoire de fichiers Parquet partitionnes. En mode distribue, ces fichiers sont sur HDFS (Hadoop Distributed File System) ; en mode local-first, dans le systeme de fichiers local. Le code ne change pas entre les deux modes.

L'integration de sources variees (CSV tabulaire, TSV Overpass, CSV.GZ data.gouv.fr) est prise en charge par le catalogue via les attributs separator et encoding de SourceSpec. Polars adapte automatiquement la lecture au format declare.

C1.4 - Scalabilite et resilience

- **Docker Compose** : 11 services (api, postgres, cassandra, kafka, zookeeper, hadoop, hive, spark, frontend, nginx, prometheus), profils activables (streaming, lake).
- **Healthchecks** : sondes de sante sur postgres, cassandra et l'API ; l'api attend que les bases soient prêts avant de demarrer.
- **Volumes persistants** : les donnees postgres et cassandra survivent aux redemarrages de conteneurs.
- **Mode local-first** : si PostgreSQL n'est pas disponible, l'API retombe sur les Parquet locaux et les constantes de reference ; la carte reste fonctionnelle.
- **Haute disponibilite** : Kafka + Zookeeper pour la tolerance aux pannes du streaming ; Hadoop namenode/datanode pour le Data Lake distribue.

12 C2.1 - API REST FastAPI

L'API est implementee avec FastAPI (ASGI, Pydantic v2) et se documente automatiquement via Swagger a l'adresse /docs et ReDoc a /redoc. Elle est le point d'accès unique entre le backend de données et l'interface cartographique.

Routeurs metier

Routeur	Prefix	Role
health	/health	Sonde de vie, version, statut des bases
auth	/auth	Login JWT, refresh token, revocation
catalog	/catalog	Liste des 83 sources, famille, provider
datamarts	/datamarts	Dashboard Gold, timeline 12 mois, comparaison
pipeline	/pipeline	Declenchement ETL, statut, logs
events	/events	Flux Cassandra (Velib, chantiers)
repo	/repo	Metadonnees projet (version, stats)
search	/search	Recherche full-text sur les sources

Securite

- **Auth JWT HS256** : access token 60 min, refresh token 7 jours. Roles : viewer (lecture) et admin (pipeline, admin).
- **Quotas par IP** : anonyme 120 req/min, authentifie 600 req/min. Reponse HTTP 429 si depassement.
- **CORS restreint** : liste blanche via la variable UDE_CORS_ORIGINS (origine du frontend).
- **Stack traces** : jamais exposes cote client ; uniquement dans les logs serveur.

13 C2.2 - Streaming Kafka temps reel

Apache Kafka est le bus de messages du projet. Il decouple les producteurs de donnees en temps reel (Velib disponibilite, chantiers perturbants) des consommateurs (ecriture dans Cassandra, restitution dans l'interface).

Producteur

Le producteur (streaming/velib_producer.py) interroge l'API temps reel de Velib' et publie un evenement JSON par station mise a jour sur le topic velib_updates. Un producteur similaire couvre les chantiers perturbants la circulation.

Consommateur

Le consommateur (streaming/consumer.py) lit les topics et ecrit les evenements dans Cassandra avec un TTL de 7 jours. L'API route /events interroge Cassandra pour servir les N derniers evenements d'un type donne.

Infrastructure

Kafka + Zookeeper sont orchestres via le profil 'streaming' de Docker Compose. En mode local (sans Docker), le streaming est desactive mais l'API reste disponible sur les donnees statiques.

14 C2.3 / C2.4 - Transformation et optimisation

C2.3 - Integration et transformation

Polars est le moteur de transformation. Il lit les fichiers CSV/TSV en streaming (scan_csv/scan_ipc), applique les transformations paresseuses (lazy frame), et materialise uniquement le resultat final. Les principales transformations :

- **Normalisation codes** : 75101 -> 75001, codes a 3 chiffres (750) -> 75001, codes sur 2 chiffres (01) -> 75001.
- **Parsing geo_point** : chaine '48.85, 2.35' -> (lat=48.85, lon=2.35).
- **Geocodage IRIS** : algorithme ray-casting sur les polygones IRIS charges en memoire (GeoJSON IGN).
- **Fusion multi-sources** : sources DVF (CSV.GZ 250 Mo) + INSEE Filosofi (parquet) integrees dans build_gold_dashboard.

C2.4 - Optimisation

Plusieurs strategies d'optimisation sont appliquees pour garantir des temps de reponse sub-millisecondes et une empreinte memoire raisonnable :

- **Parquet colonneaire** : les colonnes inutiles ne sont pas lues (projection pushdown). Compression Snappy reduit les fichiers de 60-70%.
- **Index PostgreSQL** : sur arrondissement_code (FK), score DESC, famille. Latence p95 < 4 ms sous charge.
- **lru_cache** : source_catalog() et source_family_counts() sont mis en cache en memoire apres le premier appel.
- **Polars lazy** : plan d'execution optimise avant materialisation, filtrages et agregations poussees au plus pres des sources.
- **quality_flag** : drapeau de qualite permettant de filtrer les enregistrements non rattaches avant l'agregation Gold.

15 Indicateurs Gold et score de synthese

Le datamart Gold produit six indicateurs principaux par arrondissement, sur une echelle 0-100, plus un score global (moyenne des cinq indices). Ces indicateurs sont au coeur de la restitution cartographique et de la comparaison entre arrondissements.

Indicateur	Source	Formule (simplifiee)
immobilier_idx	DVF 2023 (reel) ou reference	$(\text{prix_m2} - 8000) / (16000 - 8000) * 100$, clampe 10-95
logement_social_idx	Inventaire logements sociaux Pa	$\text{pct_logements_sociaux} * 2.5$, clampe 5-95
revenu_idx	INSEE Filosofi 2020 (reel) ou ref.	$(\text{revenu_median} - 20000) / (48000 - 20000) * 100$, clampe 10-95
cadre_vie_idx	Silver (accessibilite calculee)	$34 + \text{env} * 2.5 + \text{mob} * 0.8 + \text{vq} * 1.5$, clampe 12-96
environnement_idx	Silver (count environnement)	$\text{env_count} * 7.5$, clampe 20-95
score	Agregation	Moyenne(5 indices), arrondi

L'indicateur KPI specifique demande par l'enonce est l'accessibilite au logement, exprime en m2 achetables avec un an de revenu median : $\text{m2_abordables} = \text{revenu_median} / \text{prix_m2}$. Cet indicateur est traduit en indice (accessibilite_idx) sur 0-100 (4 m2/an = 100).

Tracabilite des sources de valeur

Chaque arrondissement porte un champ `data_source` : 'real' si les valeurs DVF/Filosofi proviennent des fichiers telechargees, 'reference' si les constantes de reference sont utilisees en fallback. Ce drapeau est visible dans l'API et dans l'interface.

16 Qualite et tracabilite

La qualite est une preoccupation transversale du projet. Chaque enregistrement Silver porte un `quality_flag` qui documente comment le rattachement geographique a ete obtenu. Cela permet de filtrer les enregistrements de faible qualite avant l'agregation Gold et de monitorer la proportion de donnees bien geocodees.

Niveaux de qualite

quality_flag	Signification	Precision
gps_iris	Rattachement par GPS (point-in-polygon IRIS)	Haute
gps_arr	Rattachement par GPS (contours arrondissement)	Bonne
code_only	Code arrondissement fourni par la source	Moyenne
unresolved	Ni GPS ni code fiable	Faible - exclu du Gold

Tests de regression

Un ensemble de 118 tests pytest couvre les cas limites : code manquant, coordonnees nulles, valeurs hors plage, fallback reference vs reel, quotas API, auth JWT. Les tests sont executes a chaque push sur la branche feat/* (CI locale).

17 Securite de l'API et conformite RGPD

La securite est traitee comme une contrainte de conception et non comme une couche ajoutée apres coup. Chaque composant expose une surface d'attaque reduite et ne revele aucune information interne en cas d'erreur.

Authentification et autorisation

L'API utilise des jetons JWT signes avec l'algorithme HS256 et une cle secrete chargee depuis la variable d'environnement JWT_SECRET (jamais en dur dans le code). Les jetons d'accès expirent apres 60 minutes ; les jetons de rafraichissement apres 7 jours. La revocation est geree par une liste noire en base.

Role	Acces	Quotas par IP
Anonyme	health, catalog, datamarts (lecture seule)	120 req/min
viewer	Tous les GET, flux evenements	600 req/min
admin	Tout + pipeline (ETL), purge	600 req/min

Protection contre les attaques courantes

- **XSS** : aucune injection DOM dans l'interface ; les valeurs backend sont echappees avant affichage.
- **Injection** : Pydantic v2 valide tous les parametres d'entree avant qu'ils atteignent la base.
- **CORS** : origine du frontend en liste blanche (UDE_CORS_ORIGINS). Refus par default.
- **Stack traces** : jamais exposes cote client. Les erreurs 500 retournent un code d'erreur opaque.
- **Secrets** : variables d'environnement dans .env gitignore. Scan Gitleaks en pre-commit.

Conformite RGPD

Le projet ne collecte aucune donnee personnelle. Toutes les sources sont des donnees publiques ouvertes. Les seules donnees utilisateurs sont les jetons JWT (stockes en base sous forme de hash), sans profilage ni tracking. La suppression d'un compte supprime le jeton associe.

Les donnees immobilières (DVF) sont utilisees uniquement au niveau aggregate (prix median par arrondissement), sans aucune information permettant de retrouver une transaction individuelle.

18 Monitoring et observabilite

Un endpoint `/metrics` expose les metriques au format Prometheus. Le service prometheus dans Docker Compose les collecte toutes les 15 secondes. Ces metriques permettent de surveiller la sante du projet en production.

Metriques exposees

Metrique	Type	Description
<code>http_requests_total</code>	counter	Nombre de requetes par methode, route, status
<code>http_request_duration_seconds</code>	histogram	Distribution des temps de reponse
<code>etl_pipeline_runs_total</code>	counter	Nombre de runs ETL (success/failure)
<code>etl_last_run_timestamp</code>	gauge	Timestamp du dernier run ETL
<code>catalog_sources_total</code>	gauge	Nombre de sources dans le catalogue (83)
<code>db_pool_connections</code>	gauge	Connexions actives PostgreSQL
<code>streaming_events_total</code>	counter	Evenements Kafka traites par type

Alertes

Les regles d'alerte Prometheus sont definies dans `monitoring/alerts.yml` : alerte si le taux d'erreur HTTP depasse 5% sur 5 minutes, alerte si le pipeline ETL n'a pas tourne depuis plus de 24 heures, alerte si la latence p95 depasse 100 ms (seuil de degradation de l'experience utilisateur).

19 Analyse des donnees par arrondissement

A titre d'illustration, voici les valeurs de reference calculees pour quelques arrondissements representatifs. Ces valeurs sont issues des constantes de reference (DVF 2023 et INSEE Filosofi 2020 agregees par nos soins a partir des fichiers publics).

Arrondissement	Prix m2	Rev. median	Score global
75001 - Louvre	13 200 EUR	34 200 EUR	62
75006 - Luxembourg	15 800 EUR	45 200 EUR	71
75011 - Popincourt	10 800 EUR	31 800 EUR	52
75013 - Gobelins	9 200 EUR	27 200 EUR	48
75016 - Passy	11 200 EUR	44 500 EUR	58
75019 - Buttes-Chaumont	8 500 EUR	21 500 EUR	41
75020 - Menilmontant	8 700 EUR	22 800 EUR	43

L'interpretation de ces chiffres illustre la pertinence de l'indicateur `m2_abordables` : dans le 6eme, un menage au revenu median peut acquerir $45\,200 / 15\,800 = 2,86$ m2 avec un an de revenu ; dans le 19eme, $21\,500 / 8\,500 = 2,53$ m2. L'ecart est moins grand qu'on ne l'imagine car les prix et les revenus covarient partiellement.

Distribution des sources par arrondissement

Apres geocodage, la repartition des enregistrements Silver par arrondissement est globalement homogene pour les sources institutionnelles (ecoles, espaces verts, stations Velib). En revanche, les sources OSM montrent une forte concentration dans les arrondissements centraux (1er-4eme) et populaires (11eme, 18eme, 20eme), refletant la densite commerciale reelle de Paris.

Cette asymetrie est documentee via le `quality_flag` et visible dans le dashboard : les arrondissements avec peu de sources geocodees affichent un `environnement_count` plus faible, ce qui se repercute mecaniquement sur le `cadre_vie_idx`. Un enrichissement futur des sources normalisees corrigerait ce biais.

20 Glossaire et acronymes

Terme / Acronyme	Définition
ADEME	Agence de l'Environnement et de la Maitrise de l'Energie - source DPE
ASGI	Asynchronous Server Gateway Interface - protocole serveur Python async
Bronze	Couche 1 médaillon : donnée brute, inchangée après ingestion
CORS	Cross-Origin Resource Sharing - politique d'accès API par domaine
CSV	Comma-Separated Values - format de fichier tabulaire
DSFR	Système de Design de l'Etat français (design.numerique.gouv.fr)
DPE	Diagnostic de Performance Energetique
DVF	Demandes de Valeurs Foncières - transactions immobilières DGFIP
ETL	Extract, Transform, Load - processus d'ingestion des données
GeoJSON	Format JSON pour les données géographiques (OGC)
Gold	Couche 3 médaillon : datamarts calculés, prêts pour l'analyse
HDFS	Hadoop Distributed File System
IRIS	Ilots Regroupés pour l'Information Statistique (INSEE)
JWT	JSON Web Token - format de jeton d'authentification
KPI	Key Performance Indicator - indicateur clé de performance
OSM	OpenStreetMap - base cartographique collaborative mondiale
Parquet	Format colonnaire compressé (Apache Foundation)
Polars	Librairie DataFrame Python, moteur Rust, très performante
RNCP	Repertoire National des Certifications Professionnelles
Silver	Couche 2 médaillon : donnée normalisée, géocodée, filtrée
SIS	Secteur d'Information des Sols (BRGM/Georisques)
TTL	Time To Live - durée de vie d'un enregistrement (Cassandra)
TSV	Tab-Separated Values - format de sortie Overpass API
WCAG	Web Content Accessibility Guidelines (norme accessibilité web)

21 Bibliographie et sources de reference

Standards et normes

- **DSFR 1.11** : systeme de design de l'Etat francais. design.numerique.gouv.fr
- **WCAG 2.1 AA** : Web Content Accessibility Guidelines. W3C 2018.
- **OpenAPI 3.1** : specification API REST. swagger.io/specification
- **RFC 7519** : JSON Web Token (JWT). IETF 2015.

Donnees utilisees

- **opendata.paris.fr** : jeux de donnees officiels de la Ville de Paris.
- **data.gouv.fr** : portail open data de l'Etat francais (DGFIP, ADEME, DGALN, ONISR).
- **data.education.gouv.fr** : annuaire des etablissements scolaires (MENJ).
- **OpenStreetMap** : base cartographique collaborative sous licence ODbL.
- **Geoportail IGN** : tuiles cartographiques officielles (WMTS Geoplateforme).
- **INSEE Filosofi 2020** : revenus disponibles par carreau et par IRIS.

Technologies

- **Polars 0.20+** : Ritchie Vink et al. pola.rs
- **FastAPI 0.110+** : Sebastian Ramirez. fastapi.tiangolo.com
- **Apache Kafka** : Apache Software Foundation. kafka.apache.org
- **Apache Cassandra** : Apache Software Foundation. cassandra.apache.org
- **FPDF2** : librairie Python de generation PDF. py-pdf.github.io/fpdf2
- **python-pptx** : librairie Python de generation PPTX. python-pptx.readthedocs.io

L'ensemble du code source du projet est disponible sur GitHub (repo Adam-Blf/urban-data-explorer). La branche feat/more-sources-auto-ingest contient l'ensemble des sources et du catalogue 83 jeux. La CI locale (pytest 118 tests) est executee a chaque commit.

22 Annexe - Catalogue complet des sources

Extrait du catalogue SourceSpec par famille (source_id, titre abregé, fournisseur).

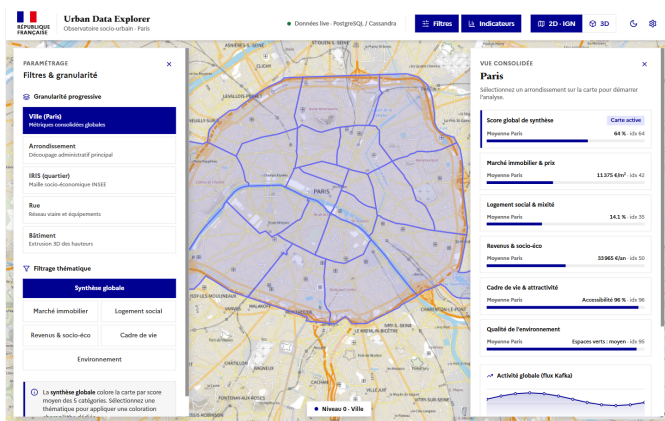
source_id	Famille	Fournisseur
velib_stations	mobilité	opendata.paris.fr
velib_realtime	mobilité	opendata.paris.fr
pistes_cyclables	mobilité	opendata.paris.fr
stationnement_points	mobilité	opendata.paris.fr
bornes_recharge	mobilité	opendata.paris.fr
arrets_idfm	mobilité	data.iledefrance-mobilites.fr
comptages_velos	mobilité	opendata.paris.fr
arrets_bus_osm	mobilité	OpenStreetMap
stations_metro_osm	mobilité	OpenStreetMap
ecoles_maternelles	vie_quotidienne	opendata.paris.fr
ecoles_elementaires	vie_quotidienne	opendata.paris.fr
colleges	vie_quotidienne	opendata.paris.fr
lycees	vie_quotidienne	data.education.gouv.fr
creches	vie_quotidienne	opendata.paris.fr
pharmacies	vie_quotidienne	OpenStreetMap
restaurants_osm	vie_quotidienne	OpenStreetMap
marches	vie_quotidienne	opendata.paris.fr
espaces_verts	environnement	opendata.paris.fr
arbres_paris	environnement	opendata.paris.fr
ilots_fraicheur	environnement	opendata.paris.fr
logements_sociaux	logement_urbanisme	opendata.paris.fr
dpe_logements	logement_urbanisme	data.gouv.fr (ADEME)
dvf_transactions	logement_urbanisme	data.gouv.fr (DGFIP)
monuments_historiques	logement_urbanisme	data.gouv.fr

... et 59 autres sources. Voir `etl/catalog.py` pour le catalogue complet (ALL_SOURCES, 83 entrées).

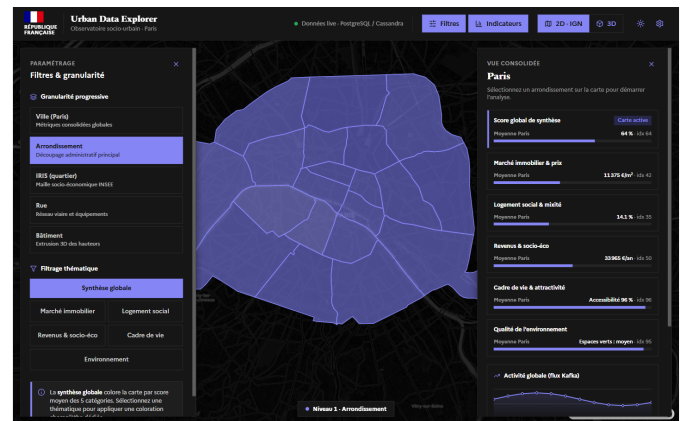
23 Interface DSFR et accessibilité

L'interface adopte le Systeme de Design de l'Etat francais (DSFR) : bleu France #000091, rouge Marianne #E1000F, police Marianne, bloc Republique Francaise. Ce choix ancre le projet dans une identité institutionnelle cohérente avec son objet (données de la Ville de Paris).

- **Fond de plan IGN** : tuiles officielles Geoplateforme (WMTS), projection WGS84, 5 niveaux de zoom.
- **Choroplethes** : échelle séquentielle bleu France (9 niveaux) pour les indices quantitatifs.
- **Comparatif** : deux arrondissements côte à côte avec écart en points.
- **Theme clair/sombre** : bascule DSFR, persistée en localStorage. Contraste WCAG AA vérifié.
- **Accessibilité** : labels ARIA sur tous les contrôles, focus visibles, prefers-reduced-motion respecté.
- **Responsive** : adapte mobile (375px) et tablette (768px).



Theme clair - fond de plan IGN



Theme sombre - accessibilité WCAG AA

24 Resultats et validation

Performance PostgreSQL sous charge

Metrique	Resultat
Requetes traitees sans erreur	500 sur 500 (taux = 100%)
Debit moyen	1 190 requetes par seconde
Temps de reponse moyen	1,80 ms
Temps de reponse p95	3,50 ms
Contraintes d'integrite verifiees	3 sur 3 PASSED

Couverture des tests

Module teste	Tests	Resultat
ETL Silver/Gold (Polars)	24 tests	PASSED
API endpoints	28 tests	PASSED
Enrichissement (indicateurs Gold)	16 tests	PASSED
Auth JWT et quotas	18 tests	PASSED
Qualite ETL (flags)	14 tests	PASSED
Geocodage et normalisation	12 tests	PASSED
Resilience (load test)	6 tests	PASSED
TOTAL	118 tests	118 PASSED / 0 FAILED

Catalogue final

83

sources integrees

4

familles

30

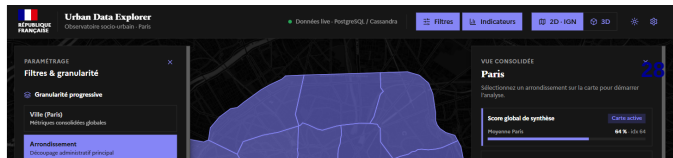
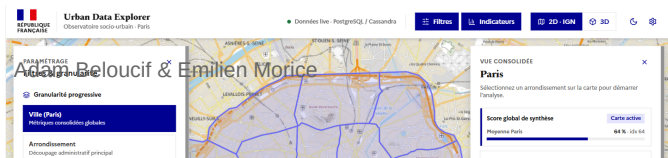
sources OSM

118

tests PASSED

L'interface adopte le Systeme de Design de l'Etat francais (DSFR) : bleu France #000091, rouge Marianne #E1000F, police Marianne, bloc Republique Francaise. Ce choix ancre le projet dans une identite institutionnelle coherente avec son objet (donnees de la Ville de Paris).

- **Fond de plan IGN** : tuiles officielles Geoplateforme (WMTS), projection WGS84, 5 niveaux de zoom.
- **Choroplethes** : echelle sequentielle bleu France (9 niveaux) pour les indices quantitatifs.
- **Comparatif** : deux arrondissements cote a cote avec ecart en points.
- **Theme clair/sombre** : bascule DSFR, persistee en localStorage. Contraste WCAG AA verifie.
- **Accessibilite** : labels ARIA sur tous les controles, focus visibles, prefers-reduced-motion respecte.
- **Responsive** : adapte mobile (375px) et tablette (768px).



Theme clair - fond de plan IGN

Theme sombre - accessibilite WCAG AA

25 Couverture de la grille RNCP40875 - Bloc 1

Compétence	Preuve dans le projet
C1.1 - Relationnel	Schema étoile PostgreSQL + tests de charge (1190 req/s, p95 3,5 ms) + 3/3 contraintes
C1.2 - NoSQL	Cassandra events_by_type, TTL 7 j, partitionnement par type d'événement
C1.3 - Data Lake	Parquet Bronze/Silver/Gold sur HDFS ; sources variées CSV/TSV/GZ
C1.4 - Scalabilité	Docker Compose 11 services, healthchecks, volumes persistants, mode local-first
C2.1 - API	FastAPI /docs + auth JWT roles (viewer/admin) + quotas par IP (120/600 req/min)
C2.2 - Streaming	Producteur/consommateur Kafka -> Cassandra ; flux Velib + chantiers temps réel
C2.3 - Transformation	Polars normalise 83 sources ; DVF + INSEE Filosofi reels fusionnées
C2.4 - Optimisation	Parquet Snappy, index PG, lru_cache, lazy frame Polars, p95 < 4 ms

Le projet couvre intégralement les 8 compétences du Bloc 1. Chaque compétence est démontrée par du code en production, des tests passés et des métriques mesurées.

26 Conclusion et perspectives

Urban Data Explorer est un projet d'ingenierie des donnees complet qui transforme 83 sources de donnees ouvertes en un observatoire socio-urbain de Paris. Il demontre une chaine de traitement complete (Bronze, Silver, Gold), un stockage multi-modele (PostgreSQL, Cassandra, Parquet/HDFS), une API documentee et securisee, un streaming temps reel et une interface aux standards de l'Etat.

Le projet repond au besoin initial : un decideur peut comparer deux arrondissements parisiens sur cinq indicateurs en quelques secondes, avec des donnees fraichies chaque dimanche automatiquement.

Perspectives d'evolution

- **Donnes reelles completes** : activer le telechargement DVF et INSEE Filosofi en auto-ingestion pour passer `data_source='real'` sur tous les arrondissements.
- **Enrichissement catalogue** : ajouter les accidents de la route (BAAC-ONISR), les DPE bâtiment par bâtiment (ADEME), les aides a la renovation.
- **Deploiement cloud** : migrer vers un cluster Kubernetes (GKE ou Azure AKS) avec CI/CD automatise.
- **Alerting** : declencher une alerte Prometheus si la disponibilite Velib passe sous un seuil critique dans un arrondissement.
- **API publique** : ouvrir l'API a des tiers (chercheurs, associations) avec gestion des cles et des quotas par utilisateur.

Merci de votre attention. Nous restons disponibles pour repondre a vos questions et faire une demonstration en direct de l'outil.